САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

«Реализация REST API на основе boilerplate»

Выполнил:
Люсин Дмитрий
Группа: К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Цель лабораторной работы - реализовать REST API на основе boilerplate с учетом ранее выполненного проектирования базы данных и технического дизайна API. В рамках работы необходимо реализовать модели, представления и контроллеры, а также продемонстрировать полностью работающее API согласно выбранному варианту.

Вариант предметной области - приложение для бронирования столиков в ресторанах. Пользователь должен иметь возможность зарегистрироваться, войти в систему, просматривать рестораны, фильтровать их по параметрам, смотреть меню, фотографии, столики, создавать бронирования, просматривать историю бронирований и оставлять отзывы.

Ход работы

Для реализации был использован boilerplate-подход. Под boilerplate в данной работе понимается базовый каркас серверного приложения, в котором заранее выделены основные части проекта: конфигурация, модели, контроллеры, маршруты, middleware, представления для формирования JSON-ответов и документация API.

В качестве технологического стека были использованы Node.js, Express, Sequelize, SQLite, JWT и Swagger. SQLite выбран как локальная база данных, поскольку она позволяет быстро запускать и демонстрировать проект без отдельной установки PostgreSQL или MySQL. При этом структура проекта остается переносимой на другую SQL-СУБД.

Компонент	**Назначение**
Node.js	Среда выполнения JavaScript на стороне сервера.
Express	Фреймворк для реализации HTTP-сервера и REST API.
Sequelize	ORM для описания моделей и взаимодействия с SQL-базой данных.
SQLite	Локальная база данных в виде файла для упрощенного запуска проекта.
JWT	Механизм авторизации пользователей через токены.
Swagger / OpenAPI	Документация и визуальная проверка реализованных эндпоинтов.

Структура проекта была разделена по MVC-подходу. Модели описывают сущности базы данных, контроллеры содержат логику обработки запросов, маршруты связывают URL-адреса с контроллерами, а представления используются как сериализаторы JSON-ответов.

```
restaurant-booking-api/
├── src/
│   ├── config/
│   ├── controllers/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   ├── utils/
│   ├── views/
│   ├── app.js
```

```
│   ├── server.js
│   └── seed.js
├── docs/
│   └── openapi.yaml
├── requests.http
├── package.json
├── .env.example
└── README.md
```

Реализация моделей

Модели были реализованы в папке src/models. Они соответствуют ERD-диаграмме, разработанной в первом домашнем задании. Каждая модель описывает отдельную таблицу базы данных и участвует в связях с другими сущностями.

Модель	Таблица БД	Назначение
User	users	Хранение пользователей, ролей и данных для входа.
Restaurant	restaurants	Хранение информации о ресторанах.
Cuisine	cuisines	Хранение типов кухонь.
RestaurantCuisine	restaurant_cuisines	Промежуточная таблица для связи ресторанов и кухонь.
RestaurantPhoto	restaurant_photos	Фотографии ресторанов.
MenuCategory	menu_categories	Категории меню конкретного ресторана.
MenuItem	menu_items	Позиции меню внутри категории.
RestaurantTable	restaurant_tables	Столики ресторана, доступные для бронирования.
Booking	bookings	Бронирования пользователей.
Review	reviews	Отзывы пользователей о ресторанах.

Реализация контроллеров

Контроллеры были реализованы в папке src/controllers. Они принимают запросы от маршрутов, обращаются к моделям, выполняют проверки и возвращают подготовленные ответы клиенту.

Контроллер	Назначение
auth.controller.js	Регистрация, вход и получение текущего пользователя.
user.controller.js	Просмотр и изменение профиля пользователя.
restaurant.controller.js	Получение, фильтрация, создание, редактирование и удаление ресторанов.
cuisine.controller.js	Управление типами кухонь.
photo.controller.js	Получение, добавление и удаление фотографий ресторанов.
menu.controller.js	Получение меню ресторана.
table.controller.js	Получение и управление столиками ресторана.
booking.controller.js	Создание, просмотр, отмена и изменение статуса бронирований.
review.controller.js	Получение, создание, редактирование и удаление отзывов.

Реализация представлений

Так как приложение реализует REST API, представления не являются HTML-страницами. В данной работе views используются как сериализаторы JSON-ответов. Они отделяют внутреннюю структуру моделей от данных, которые возвращаются клиенту.

Представление	Назначение
user.view.js	Формирует безопасный JSON пользователя без password_hash.
restaurant.view.js	Формирует данные ресторана для ответа API.
cuisine.view.js	Формирует данные кухни.
photo.view.js	Формирует данные фотографии ресторана.
menu.view.js	Формирует структуру меню и блюд.
table.view.js	Формирует данные столиков.
booking.view.js	Формирует данные бронирований.
review.view.js	Формирует данные отзывов.

Маршруты и основные эндпоинты

Маршруты были реализованы в папке src/routes. Они группируют эндпоинты по предметным областям: авторизация, пользователи, рестораны, кухни, меню, столики, бронирования и отзывы.

Группа	Примеры эндпоинтов	Назначение
Auth	POST /api/auth/register POST /api/auth/login GET /api/auth/me	Регистрация, вход, получение текущего пользователя.
Restaurants	GET /api/restaurants GET /api/restaurants/{id}	Поиск, фильтрация и просмотр ресторанов.
Menu	GET /api/restaurants/{id}/menu	Получение меню ресторана.
Photos	GET /api/restaurants/{id}/photos	Получение фотографий ресторана.
Tables	GET /api/restaurants/{id}/tables	Получение столиков ресторана.
Bookings	POST /api/bookings GET /api/bookings/my	Создание и просмотр бронирований.
Reviews	GET /api/restaurants/{id}/reviews POST /api/restaurants/{id}/reviews	Получение и создание отзывов.

Авторизация, роли и обработка ошибок

В проекте реализована JWT-авторизация. После входа пользователь получает токен, который передается в защищенные эндпоинты. Для разделения доступа используются роли user и admin. Например, обычный пользователь может создавать бронирования и отзывы, а административные действия по управлению ресторанами, меню и столиками доступны только пользователю с ролью admin.

Также реализованы middleware для проверки авторизации, проверки роли и централизованной обработки ошибок. API возвращает стандартные HTTP-коды: 400 для некорректных данных, 401 для отсутствия авторизации, 403 для недостатка прав, 404 для

отсутствующего ресурса и 409 для конфликтов, например повторного бронирования одного столика на одно и то же время.

Вывод

В результате лабораторной работы был реализован рабочий монолитный REST API для приложения бронирования столиков в ресторанах. Проект построен на основе boilerplate и разделен на модели, контроллеры, маршруты, middleware и представления-сериализаторы.

Реализованные модели соответствуют ранее спроектированной базе данных. Контроллеры обеспечивают основную бизнес-логику приложения: регистрацию, авторизацию, поиск ресторанов, просмотр меню, фотографий и столиков, создание бронирований и отзывов. Представления формируют корректные JSON-ответы и скрывают служебные данные, например хеш пароля пользователя.

Таким образом, лабораторная работа показывает переход от проектирования базы данных и OpenAPI-схемы к работающему backend-приложению, которое можно запускать локально, тестировать через Swagger и использовать как основу для дальнейшего разделения на микросервисы.